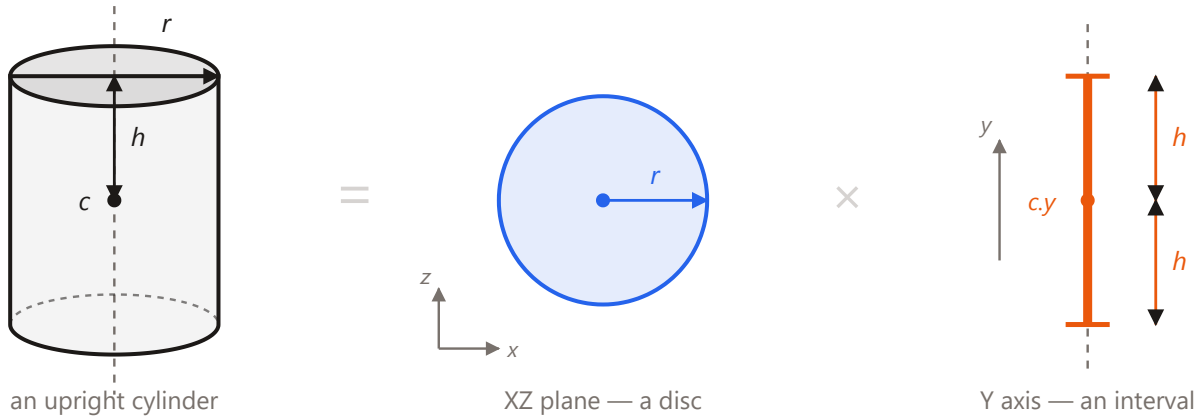


An upright cylinder is two independent tests

Everything downstream depends on this one property.



A cylinder whose axis is locked to Y factors into a **disc in XZ** and an **interval in Y**. A point is inside the cylinder if and only if *both* hold — and the two tests never consult each other.

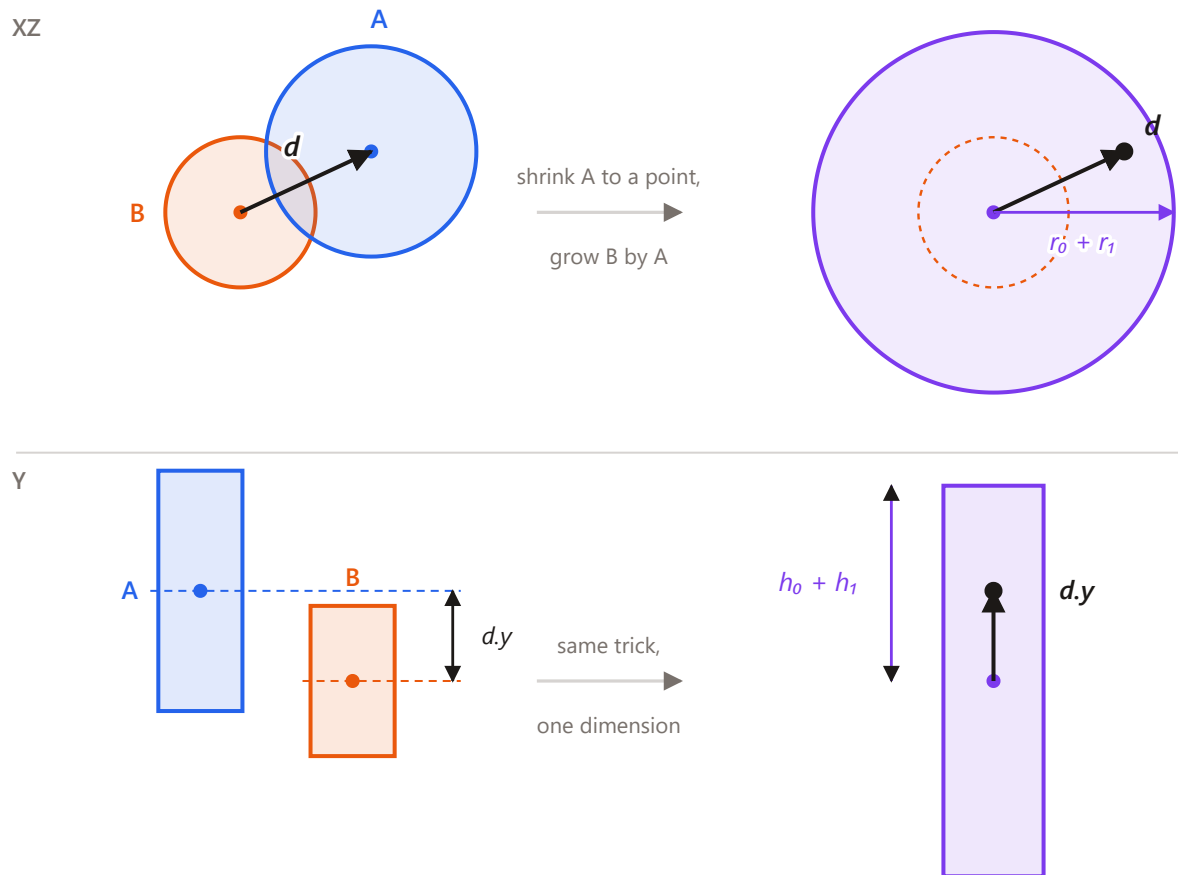
$$p \in \text{cylinder} \Leftrightarrow p.xz \in \text{Disc}(c.xz, r) \quad \text{and} \quad p.y \in \text{Interval}(c.y \pm h)$$

That independence is the whole reason the rest of this is closed-form. It survives only while the axis stays upright: tilt either cylinder and the disc smears into an ellipse whose shape depends on the tilt, the factorization dies, and you are back to iterating with GJK/EPA.

This is why *upright cylinders* is a locked decision and not a simplification to revisit later — the exactness of every formula on the following pages is downstream of it.

Minkowski sum: two cylinders become one

Shrink A to a point, grow B by A. Shape-vs-shape becomes point-in-shape.



Rather than test two moving shapes, collapse A to a single point and inflate B by A's shape. The inflated shape is the **configuration-space obstacle**, and the question becomes whether the centre-to-centre vector d lands inside it.

$$(D_0 \times I_0) \oplus (D_1 \times I_1) = (D_0 \oplus D_1) \times (I_0 \oplus I_1)$$

Disc $\oplus$ disc

A disc of radius $r_A + r_B$ — the familiar result that two circles overlap exactly when their centres are closer than the sum of their radii.

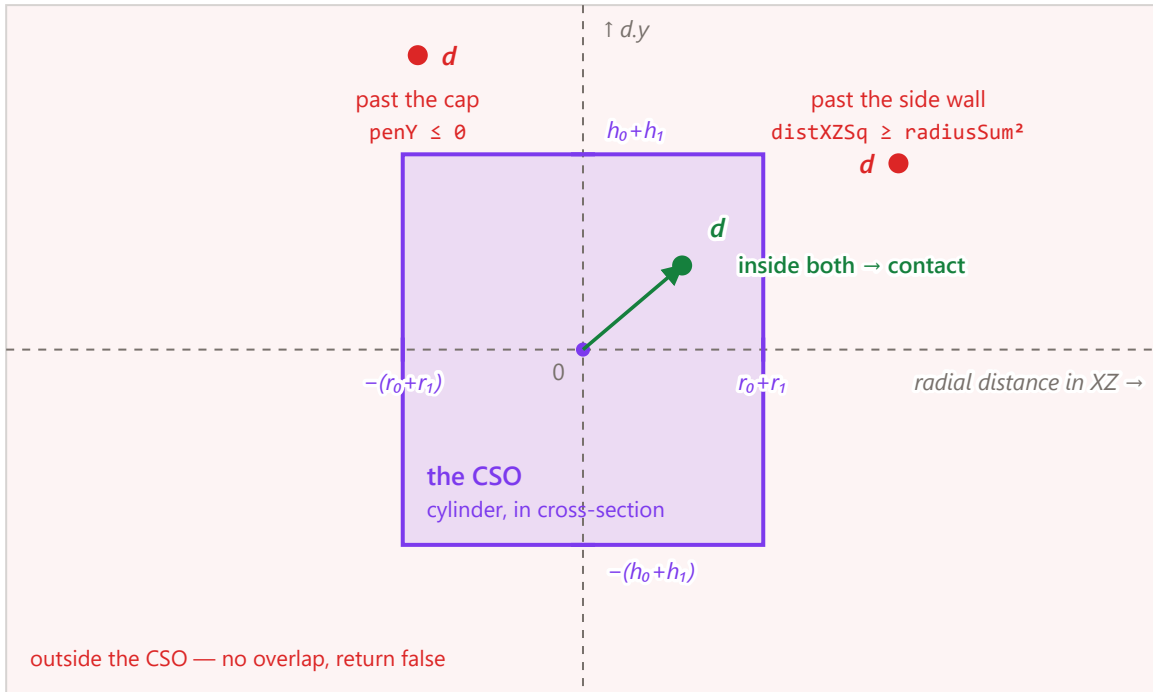
Interval $\oplus$ interval

An interval of half-length $h_A + h_B$. Sum the half-heights, keep the axis.

Because the sum distributes over that product, the CSO is **another upright cylinder**: radius $r_A + r_B$, half-height $h_A + h_B$, centred at the origin of d . No approximation entered anywhere.

Point in cylinder — two early-outs

The cross-section of the CSO, with d plotted against it.



```
// Radial test in XZ. Both penetration depths below are exact -- the
// configuration-space obstacle is a cylinder of radius rA+rB and
// height hA+hB, so there is nothing to iterate toward.
float radiusSum = a.Radius + b.Radius;
float distXZSq = d.x * d.x + d.z * d.z;
if (distXZSq >= radiusSum * radiusSum) return false; // past the side wall

float penY = (a.HalfHeight + b.HalfHeight) - Mathf.Abs(d.y);
if (penY <= 0f) return false; // past the caps
```

One early-out per factor. `Mathf.Abs(d.y)` because the CSO is symmetric about zero — it does not care whether A sits above or below B.

Why compare squares

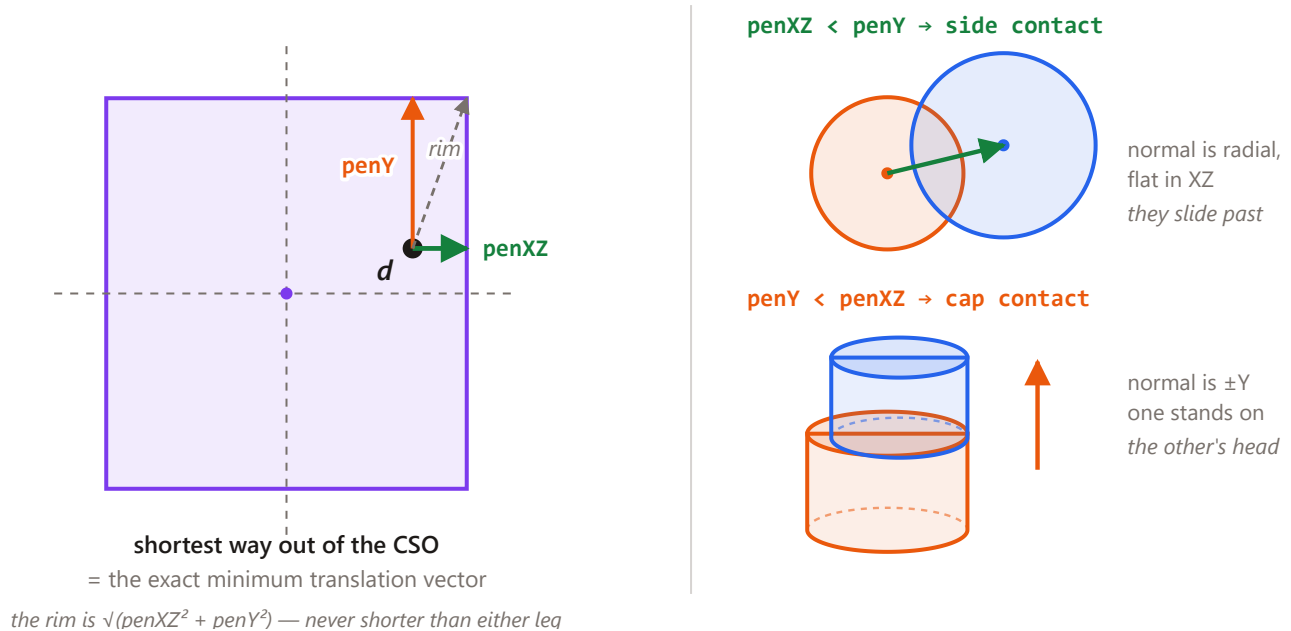
$\sqrt{\cdot}$ is monotonic, so $\sqrt{u} \geq \sqrt{v} \Leftrightarrow u \geq v$. Comparing squares gives the identical answer and skips the square root on the rejection path — the common path. The `Sqrt` is paid only after the test passes, where the normal needs it anyway.

Why `>=` and not `>`

Exact touching counts as *not* colliding. With `>`, tangent cylinders emit a contact of depth 0 — a constraint that does no work, consumes a solver iteration, and feeds a zero-length vector into the normalization below.

The shortest exit, and why it is exact

Distance from an interior point to the boundary of a cylinder.



For a point inside at radial distance ρ and height y , there are only two candidate exits — and a third that never wins.

penXZ = $R - \rho$ out through the side wall
 penY = $H - |y|$ out through the nearer cap
 to the rim: $\sqrt{penXZ^2 + penY^2}$

The rim distance is a hypotenuse, so it is $\geq$ **both legs** — always. The nearest boundary point is therefore on the wall or on a cap, never on the rim, and $\min(penXZ, penY)$ is the true minimum translation vector.

```
if (penY < penXZ) { // cap contact: normal is straight up or down
    contact.Normal = new Vector3(0f, d.y >= 0f ? 1f : -1f, 0f);
    contact.Depth = penY;
} else if (distXZ > Epsilon) { // side contact: along the horizontal axis
    contact.Normal = new Vector3(d.x / distXZ, 0f, d.z / distXZ);
    contact.Depth = penXZ;
} else { // centres exactly stacked -- pick a stable arbitrary direction
    contact.Normal = Vector3.right;
    contact.Depth = penXZ;
}
```

This is what GJK/EPA exist to approximate for arbitrary convex shapes: EPA expands a polytope *iteratively* toward the CSO surface, converging to within a tolerance. Here the CSO is a cylinder you can write down, so the depth is a subtraction and the branch is a comparison — no tolerance, no iteration count, no convergence failure.